

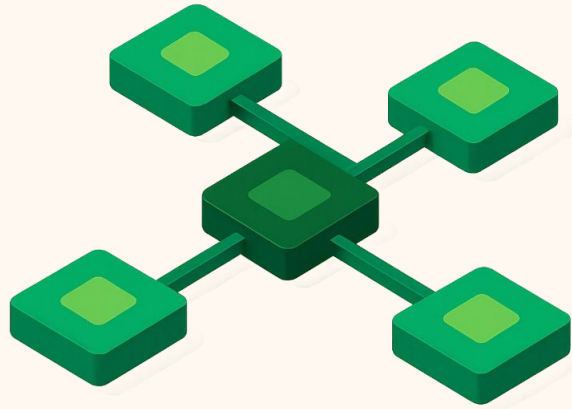


React Hooks

Modern state management and side effects for functional components

Notes by Sir Yasir Ali Lashari

What Are React Hooks?



Hooks are built-in functions in React that allow functional components to use state management and lifecycle features. They eliminate the need for class components and make code more modular and reusable.

Core Capabilities of Hooks

Hooks enable functional components to handle complex React features that were previously exclusive to class components.

01

State Management

Add local state to functional components using `useState` and `useReducer`

03

DOM Interaction

Access and manipulate DOM elements directly using `useRef`

02

Side Effects

Handle data fetching, subscriptions, and DOM manipulation with `useEffect`

04

Logic Separation

Separate concerns better than class components ever could

Lifecycle in React

Lifecycle refers to the different phases of a component's existence:



Mounting

Component is created and added to the DOM



Updating

Component re-renders due to state or prop changes



Unmounting

Component is removed from the DOM

Lifecycle handling is used for:

- Data fetching
- Event listeners
- Logging or tracking changes
- Cleanup operations



State in React

State refers to data that belongs to a component and can change over time. When state changes, the component re-renders to reflect updated values in the UI.

Key characteristics:

-  Mutable data managed within a component
-  Triggers re-render on update
-  Used for dynamic behavior in applications

Example use cases:



Counters



Toggle buttons



Form inputs



Dynamic UI updates

useState: Managing Component State

The useState Hook

useState is a React Hook used to declare and manage state variables in functional components.

How It Works

`count` stores the current state value

`setCount` updates the state

```
import { useState } from "react";

function My_useState(){
  const [count ,setCount]=useState(0);

  return(
    <>
    <h1>Count : {count}</h1>

    <button onClick={() => setCount(count + 1)}>Increase</button>
    <button onClick={() => setCount(count - 1)}>Decrease</button>
    </>)
  }

export default My_useState;
```

useEffect: Side Effects

useEffect is a React Hook used to perform side effects in functional components. It is commonly used to handle lifecycle-related operations.

Side effects include:

- API calls
- Logging
- Subscriptions
- DOM updates

```
import { useEffect, useState } from "react";

function My_useEffect() {
  const [count, setCount] = useState(0);

  // runs when count changes
  useEffect(() => {
    console.log("Count changed:", count);
  }, [count]);

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Click
      </button>
    </div>
  );
}
```

```
export default My_useEffect;
```

useRef Hooks

What useRef Does

useRef is a React Hook used to persist values across renders without causing re-rendering. It is also used to directly access DOM elements.

Common Use Cases

- Focusing input elements programmatically
- Storing timer IDs
- Keeping track of previous values
- Accessing canvas or video elements

```
import { useState, useRef, useEffect } from "react";

function My_useRef() {
  const [count, setCount] = useState(0);

  const prevCount = useRef(); // yahan previous value store hogi

  useEffect(() => {
    prevCount.current = count; // current value ko store kar liya
  }, [count]);

  return (
    <div>
      <h2>Current: {count}</h2>
      <h2>Previous: {prevCount.current}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```